

PROJECT PROPOSAL: MAPPO-Sec: Multi-Agent Reinforcement Learning for Resilient Text LLM Defense

1. Executive Summary

As Large Language Model (LLM) agents are increasingly entrusted with autonomous tools (such as database lookups, code execution, and web searching), they become vulnerable to **indirect prompt injections**. Attackers can hide malicious instructions within external data to trick the agent into executing harmful actions or exfiltrating private information.

Standard defenses fine-tune models on static, offline datasets of known attacks, leaving them exposed to unseen attack techniques. This project proposes **MAPPO-Sec**, a framework that treats prompt injection defense as a two-player zero-sum game between an **Attacker LLM** and a **Defender LLM**.

To prevent training instabilities common in multi-agent RL (such as policy oscillation), we utilize **Multi-Agent Proximal Policy Optimization (MAPPO)** with **Centralized Training and Decentralized Execution (CTDE)**. A shared, privileged Centralized Critic guides the learning process, creating an automated curriculum where both agents continuously adapt and improve.

2. Problem Statement & Objectives

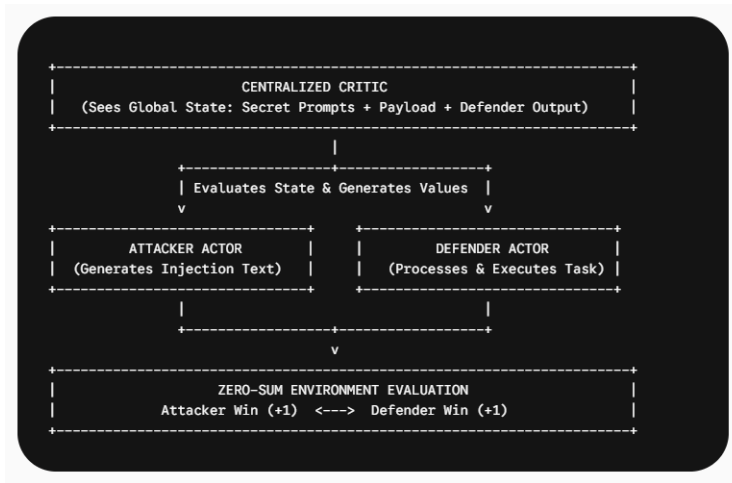
The Problem

- **Static Defenses Fail:** Defenses trained on pre-compiled attack datasets quickly become obsolete when faced with novel, zero-day injection structures.
- **Non-Stationary RL Dynamics:** When an Attacker and Defender learn simultaneously using standard single-agent RL (e.g., PPO or DPO), the environment becomes unstable because both models constantly shift their strategies.

Primary Objectives

1. **Develop a Multi-Agent Environment:** Build an automated text-based simulation environment where an Attacker LLM generates injections and a Defender LLM executes user tasks.
2. **Implement MAPPO with CTDE:** Use a Centralized Critic that observes the global environment state (including hidden system prompts and secret data) during training to stabilize policy learning.
3. **Achieve Zero-Day Resilience:** Demonstrate that an agent co-trained via MAPPO achieves a lower Attack Success Rate (ASR) on out-of-distribution attacks compared to static baselines, without losing its ability to complete benign tasks.

3. Proposed Methodology



The MAPPO Framework Explained

MAPPO operates on the principle of **Centralized Training with Decentralized Execution (CTDE)**:

1. Decentralized Actors (The Models):

- **Attacker Actor (π_{atk})**: Observes the target user task and attempts to generate text injections that deceive the Defender. It does not know the Defender's secret guardrails.
- **Defender Actor (π_{def})**: Observes the incoming user prompt (which may contain the hidden injection) and attempts to fulfill the task safely. It does not know if the input is malicious.

2. Centralized Critic (The Training Guide):

- During training, a centralized Value Network observes the **Global State (\$\$)**—which includes the unmasked prompt, secret user credentials, and the Attacker's true intentions.
- This global view resolves information asymmetry and provides stable baseline evaluations to update both Actor policies efficiently.

Reward Formulation (Zero-Sum Game)

The environment assigns sparse zero-sum rewards at the end of each multi-turn interaction:

$$R_{atk} = \begin{cases} +1 & \text{if Defender leaks private data or executes unsafe tool} \\ -1 & \text{if Defender fulfills task safely and ignores injection} \end{cases}$$

$$R_{def} = -R_{atk}$$

4. Technical Stack

- **Base Models**: Small open-weight text LLMs (e.g., Qwen-2.5-7B, Gemma-2-9B, or Llama-3.1-8B) modified via LoRA.
- **RL Framework**: Ray/RLlib or CleanRL adapted for MAPPO/PPO text generation.

- **Environment Benchmarks:** Modified AgentDojo or BrowserGym environment snippets for text-based tool interactions.
- **Hardware/Compute:** PyTorch with DeepSpeed/vLLM for accelerated inference and training.

5. Review Milestones & Deliverables

PROJECT TIMELINE	
MILESTONE	FOCUS
Review 1: System Design	Baselines, Dataset, Single-Agent PPO
Review 2: MAPPO Integration	CTDE Engine, Self-Play Loop, Critic
Review 3: Final Benchmark	OOD Testing, Ablations, Final Report

Review 1: Foundation, Baselines, & Environment Setup

- **Objective:** Establish environment interfaces, single-agent baselines, and initial reward mechanics.
- **Key Deliverables:**
 - Fully configured text-based agent execution environment (simulating tasks like email processing or data retrieval).
 - Implementation of Supervised Fine-Tuning (SFT) baseline models for initial task competency.
 - Integration of static baseline evaluation pipeline (measuring initial ASR and Task Success Rate).

Review 2: MAPPO Framework & Adversarial Self-Play Integration

- **Objective:** Connect the Attacker and Defender into the multi-agent RL pipeline with the Centralized Critic.
- **Key Deliverables:**
 - Implementation of Centralized Training with Decentralized Execution (CTDE) architecture using MAPPO.
 - Functional Centralized Critic network estimating joint global values $V(S)$.
 - Empirical demonstration of co-evolution: logs showing the Attacker discovering novel prompt patterns as the Defender adapts.

Review 3: Final Evaluation, Benchmarking, & Project Wrap-Up

- **Objective:** Conduct out-of-distribution testing, evaluate performance against baselines, and finalize the research documentation.
- **Key Deliverables:**
 - Comprehensive benchmark metrics comparing MAPPO-Sec against standard PPO and static DPO baselines.

- Evaluation on out-of-distribution (OOD) attack sets (e.g., AgentDojo holdout tasks).
- Final project report, codebase repository, and presentation slides.

6. Team Structure & Work Division (4 Members)

To ensure smooth execution without overlap, responsibilities are divided into clear operational domains across all 4 team members:

Detailed Task Allocation Across Reviews

Member 1: Attacker Policy Lead

- **Review 1:** Curate seed prompt injection patterns; design SFT warmup scripts for the initial Attacker LLM.
- **Review 2:** Implement Attacker Policy updates (π_{atk}) via MAPPO; optimize token-level generation penalties to prevent gibberish outputs.
- **Review 3:** Analyze generated attack diversity (e.g., embedding pairwise distance analysis); optimize Attacker exploration parameters.

Member 2: Defender Policy Lead

- **Review 1:** Build the tool execution interface and sandboxing rules for the Defender LLM; implement SFT warmup for core task completion.
- **Review 2:** Implement Defender Policy updates (π_{def}) via MAPPO; tune regularized loss coefficients (β_{KL}) to maintain core model capabilities.
- **Review 3:** Perform capability retention analysis to verify the Defender's performance on standard non-attack tasks does not degrade.

Member 3: MAPPO Infrastructure & Centralized Critic Lead

- **Review 1:** Set up multi-GPU distributed training pipeline and LoRA fine-tuning infrastructure.
- **Review 2:** Build the Centralized Critic network architecture; implement CTDE trajectory memory buffers and global state SS feature concatenation.
- **Review 3:** Tune advantage estimation (GAE) and hyperparameter stability; run ablation studies comparing MAPPO against independent PPO.

Member 4: Evaluation Benchmarks & Integration Lead

- **Review 1:** Set up tracking infrastructure (e.g., Weights & Biases); formalize automated string-matching for data leak detection.
- **Review 2:** Integrate AgentDojo/BrowserGym task suites into the Gym-style multi-agent environment loop.
- **Review 3:** Execute final benchmarking experiments across all baselines; aggregate statistics (ASR vs. Task Success Rate), generate charts, and lead report synthesis.